

Gratitude

Server host procedure

RHEL VM deploy checklist

October 1, 2026

This document covers how to host Gratitude on a Red Hat Enterprise Linux (RHEL) server (RHEL 8 or 9 and compatible clones such as Rocky or Alma unless noted). It assumes SSH access (IP address, OS, username, and a password or key).

The last section is the Azure AD work-account setup: sign-in, the company recipient list, and sending a card from the signed-in mailbox. Leave that section unused until the identity team sends the application identifiers.

Contents

1	What you are hosting	2
2	Access the server	2
3	Install base software (RHEL)	2
3.1	Node.js 20+	2
3.2	PostgreSQL	3
3.3	Firewall (firewalld)	3
4	Deploy the application	3
4.1	Clone and install	3
4.2	API environment	3
4.3	Database migrate and seed	4
4.4	Build the web UI	4
4.5	Keep uploads on disk	4
5	Run the API as a service	4
6	nginx reverse proxy	5
6.1	TLS certificate	5
6.2	SELinux	6
7	CI and CD	6
8	Smoke checks	6

9 Azure AD sign-in, directory, and mail	7
9.1 What to ask the identity team	7
9.2 Where to put the values	7
9.3 Network	8
9.4 Check that it works	8
9.5 If sign-in fails	9

1 What you are hosting

Gratitude needs two parts on the server:

1. API — Node.js app in `api/` (port 3001 by default). Serves JSON, auth, and uploaded card images under `/uploads`.
2. Web — built React app from `web/`. Served as static files with nginx.

Also required: PostgreSQL, and durable disk for `api/uploads/`.

One VM can host the API, the web files, and PostgreSQL. Use nginx to route by hostname if several apps share the VM.

2 Access the server

1. Confirm you have the public IP, SSH username, and a password or SSH key.
2. On RHEL, try usernames in this order if IT did not specify one: `root`, `cloud-user`, `admin`.
3. From Windows PowerShell: `ssh USERNAME@SERVER_IP`
4. Confirm the OS: `cat /etc/os-release`
5. Change a shared password on first login if it was emailed to you.
6. Prefer SSH keys for ongoing access.

3 Install base software (RHEL)

Use `dnf` (not `apt`).

```
sudo dnf -y update
sudo dnf -y install git curl tar gcc-c++ make nginx firewalld
sudo systemctl enable --now firewalld
```

3.1 Node.js 20+

```
curl -fsSL https://rpm.nodesource.com/setup_20.x | sudo bash -
sudo dnf -y install nodejs
node -v
npm -v
```

If corporate policy blocks NodeSource, install Node from the approved internal RPM channel. Target Node 20 or newer.

3.2 PostgreSQL

```
sudo dnf -y module reset postgresql
sudo dnf -y module enable postgresql:16
sudo dnf -y install postgresql-server postgresql
sudo postgresql-setup --initdb
sudo systemctl enable --now postgresql
```

If `postgresql:16` is unavailable, enable the highest version the RHEL release supports.

Create the app role and database (replace the password):

```
sudo -u postgres createuser grat -P
sudo -u postgres createdb project_grat -O grat
```

Allow local password auth for the app user. Edit `pg_hba.conf` (often under `/var/lib/pgsql/data/`) so local TCP uses `scram-sha-256` or `md5`, then `sudo systemctl restart postgresql`.

3.3 Firewall (firewalld)

The network team allows inbound TCP 22 (SSH) and TCP 443 (HTTPS) only. Do not open port 80.

```
sudo firewall-cmd --permanent --add-service=ssh
sudo firewall-cmd --permanent --add-service=https
sudo firewall-cmd --reload
sudo firewall-cmd --list-all
```

Do not expose port 3001. nginx on this host proxies it on 443. PostgreSQL stays on the server. Do not open port 5432 to user PCs.

4 Deploy the application

4.1 Clone and install

```
sudo mkdir -p /var/www
cd /var/www
sudo git clone https://github.com/SeniruR/project_grat.git
sudo chown -R $USER:$USER /var/www/project_grat
cd /var/www/project_grat
npm install
cd api && npm install
cd ../web && npm install
```

4.2 API environment

Create `/var/www/project_grat/api/.env`. Start with dev sign-in. Switch to Azure AD only after section 9.

```
DATABASE_URL="postgresql://grat:PASSWORD@127.0.0.1:5432/project_grat?schema=public"
PORT=3001
HOST=0.0.0.0
```

```
JWT_SECRET="REPLACE_WITH_LONG_RANDOM_SECRET"
CORS_ORIGIN="https://YOUR_WEB_HOSTNAME"
PUBLIC_API_URL="https://YOUR_API_HOSTNAME"
AUTH_MODE=dev
DIRECTORY_MODE=mock
MAIL_MODE=mock
```

PUBLIC_API_URL must be the API public URL. CORS_ORIGIN must include the browser origin of the web UI.

4.3 Database migrate and seed

```
cd /var/www/project_grat/api
npx prisma generate
npx prisma migrate deploy
npm run db:seed
```

Seed loads demonstration users. Skip seed on a real production database if you do not want those accounts. The Azure AD column is added by **migrate deploy**; run it before anyone signs in with a work account.

4.4 Build the web UI

```
cd /var/www/project_grat/web
export VITE_API_URL="https://YOUR_API_HOSTNAME"
npm run build
```

Output is in web/dist/.

4.5 Keep uploads on disk

```
mkdir -p /var/www/project_grat/api/uploads
```

Back up this folder. If it is lost, card images break while database rows remain.

5 Run the API as a service

```
sudo useradd -r -s /sbin/nologin grat || true
sudo chown -R grat:grat /var/www/project_grat
```

Create /etc/systemd/system/gratitude-api.service:

```
[Unit]
Description=Gratitude API
After=network.target postgresql.service

[Service]
Type=simple
User=grat
Group=grat
```

```
WorkingDirectory=/var/www/project_grat/api
EnvironmentFile=/var/www/project_grat/api/.env
ExecStart=/usr/bin/npm start
Restart=always
RestartSec=5
```

```
[Install]
WantedBy=multi-user.target
```

Then:

```
sudo systemctl daemon-reload
sudo systemctl enable --now gratitude-api
curl http://127.0.0.1:3001/health
```

6 nginx reverse proxy

On RHEL, site files go in `/etc/nginx/conf.d/`. nginx listens on 443 only. Nothing listens on port 80, so browsers must use `https://`.

Example: web on `grat.example.com`, API on `api-grat.example.com`.

6.1 TLS certificate

Port 80 is closed, so Certbot's HTTP check cannot run. Ask the server team for a certificate that covers both names (or one certificate per name). Place the files on the server, for example:

```
/etc/pki/tls/certs/gratitude.crt
/etc/pki/tls/private/gratitude.key
```

CORS_ORIGIN, PUBLIC_API_URL, and VITE_API_URL stay on `https://` URLs. Rebuild the web app if those hostnames change.

`/etc/nginx/conf.d/gratitude-web.conf`:

```
server {
    listen 443 ssl;
    server_name grat.example.com;

    ssl_certificate      /etc/pki/tls/certs/gratitude.crt;
    ssl_certificate_key  /etc/pki/tls/private/gratitude.key;

    root /var/www/project_grat/web/dist;
    index index.html;
    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

`/etc/nginx/conf.d/gratitude-api.conf`:

```

server {
    listen 443 ssl;
    server_name api-grat.example.com;

    ssl_certificate      /etc/pki/tls/certs/gratitude.crt;
    ssl_certificate_key  /etc/pki/tls/private/gratitude.key;

    client_max_body_size 20M;
    location / {
        proxy_pass http://127.0.0.1:3001;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

sudo nginx -t
sudo systemctl enable --now nginx
sudo systemctl reload nginx

```

6.2 SELinux

If nginx returns 502 or 403 with SELinux enforcing:

```

sudo setsebool -P httpd_can_network_connect 1
sudo dnf -y install polycoreutils-python-utils
sudo semanage fcontext -a -t httpd_sys_content_t \
    "/var/www/project_grat/web/dist(/.*)?"
sudo restorecon -Rv /var/www/project_grat/web/dist

```

7 CI and CD

Continuous deployment SSHs to this host and runs `scripts/deploy-rhel.sh`. GitHub secrets: `DEPLOY_HOST`, `DEPLOY_USER`, `DEPLOY_SSH_KEY`. Optional: `DEPLOY_PORT` (default 22), `DEPLOY_APP_ROOT` (default `/var/www/project_grat`).

On the server:

```

sudo mkdir -p /etc/gratitude
sudo tee /etc/gratitude/deploy.env <<'EOF'
VITE_API_URL=https://YOUR_API_HOSTNAME
EOF
sudo chmod 640 /etc/gratitude/deploy.env

```

8 Smoke checks

1. `curl https://YOUR_API_HOSTNAME/health` returns OK.

2. Open the web URL and sign in.
3. Upload or import a card and confirm images appear after refresh.
4. Restart the API and confirm images still load.

9 Azure AD sign-in, directory, and mail

This is the work-account path. It does three things with one Azure AD application:

1. Sign in with a work account.
2. Search colleagues when choosing who receives a card.
3. Send the card from that signed-in person's mailbox. A bulk send is one card per selected person, each sent as that user.

In the Azure portal the blade is named **Microsoft Entra ID**. That is the current portal name of Azure Active Directory. Create the app registration there. Gratitude calls this mode `AUTH_MODE=azure`.

Until the three identifiers below exist, keep the server on dev sign-in (`AUTH_MODE=dev`, `DIRECTORY_MODE=mock`, `MAIL_MODE=mock`). The sign-in button appears only after `AUTH_MODE=azure` and the three identifiers are set.

9.1 What to ask the identity team

Ask them to register an application named Gratitude in this organization's directory only.

- Platform: **Web** (not a single-page or mobile platform).
- Redirect URI, exactly, with no trailing slash on the host:

`https://YOUR_API_HOSTNAME/auth/azure/callback`

For a laptop test before the server name exists, also add:

`http://localhost:3001/auth/azure/callback`

- Delegated Microsoft Graph permissions, then **admin consent**:
 - `User.Read` — sign-in profile
 - `User.ReadBasic.All` — search the company directory
 - `Mail.Send` — send as the signed-in user
- A client secret. Copy the secret value when it is created. The portal will not show it again. They should give you three values from the app's Overview and Certificates pages:

Portal label	api/.env name
Directory (tenant) ID	<code>AZURE_TENANT_ID</code>
Application (client) ID	<code>AZURE_CLIENT_ID</code>
Client secret value	<code>AZURE_CLIENT_SECRET</code>

9.2 Where to put the values

Edit `/var/www/project_grat/api/.env` on the server (or `api/.env` on a laptop). Replace the three mode lines and add the Azure block:

```
AUTH_MODE=azure
DIRECTORY_MODE=graph
MAIL_MODE=graph
AZURE_TENANT_ID=paste-tenant-id
AZURE_CLIENT_ID=paste-client-id
AZURE_CLIENT_SECRET=paste-secret
AZURE_REDIRECT_URI=https://YOUR_API_HOSTNAME/auth/azure/callback
AZURE_WEB_ORIGIN=https://YOUR_WEB_HOSTNAME
AZURE_ADMIN_EMAILS=you@yourcompany.com
```

- AZURE_REDIRECT_URI must match the redirect URI on the app registration, character for character.
- AZURE_WEB_ORIGIN is the site people open in the browser. After Microsoft signs them in, the API sends them back here.
- AZURE_ADMIN_EMAILS is a comma-separated list of work emails. Those people become Gratitude admins when they sign in. Everyone else signs in as a normal sender. Designer access is still granted inside the app.
- CORS_ORIGIN must include AZURE_WEB_ORIGIN.
- PUBLIC_API_URL must be the same API host used in the redirect URI.

Apply the database column if this checkout is new to the server, then restart:

```
cd /var/www/project_grat/api
npx prisma migrate deploy
sudo systemctl restart gratitude-api
```

Rebuild the web UI if VITE_API_URL changed, then reload nginx.

On a laptop, the same block with local URLs is:

```
AZURE_REDIRECT_URI=http://localhost:3001/auth/azure/callback
AZURE_WEB_ORIGIN=http://localhost:5173
```

Restart `npm run dev` after saving `api/.env`.

9.3 Network

The server 172.25.40.175 (or whichever host runs the API) must open outbound TCP 443 to:

- login.microsoftonline.com
- graph.microsoft.com

That is separate from the PC-to-server rules (inbound TCP 443 and 22 only; port 80 stays closed). Browsers also need to reach login.microsoftonline.com, which office networks usually already allow.

PostgreSQL stays on the server. Do not open port 5432 or 3001 to user PCs.

9.4 Check that it works

1. Open the site. The sign-in panel says to use a work account. It must not ask for an employee number.
2. Sign in. You should land in Gratitude without typing a password into Gratitude itself.
3. Open a card, choose recipients, and type a colleague's name. The list comes from the company directory, not the seeded example users.

4. Send to one or more people. Each message is sent from the signed-in mailbox, and a copy is saved in that mailbox's Sent Items.
5. If search or send says consent is missing, the identity team still needs to grant admin consent for the three permissions above.

9.5 If sign-in fails

- **Redirect URI mismatch.** The value in the portal and `AZURE_REDIRECT_URI` differ. Fix one of them and restart the API.
- **Azure AD details are not in the API yet.** `AUTH_MODE=azure` is set but one of the three identifiers is empty.
- **Account is not in the configured tenant.** The person signed in with a personal Microsoft account or another company. Use the work account.
- **Need admin consent.** Directory search or send returns a permissions error. Ask the identity team to grant admin consent again.
- **API cannot reach Microsoft.** From the server, `curl -I https://login.microsoftonline.com` must connect. If it does not, request outbound TCP 443.